Spécialisation Data Scientist (Parcours IA)

Projet de Fin de Formation

Scoring de Défaut de Crédit + POC GenAI

Talk to my Data — LangChain v1 + Streamlit

Contexte & Mission

Vous intégrez la direction **"Recouvrement & Risque"** d'une banque de détail.

Mission ML

Construire un modèle de prédiction du défaut de carte de crédit.

Défi majeur : Prendre en compte de manière rigoureuse le déséquilibre des classes (imbalanced dataset).

Mission GenAI

Réaliser un POC **« Talk to my Data »**.

Développer un assistant conversationnel piloté en langage naturel permettant d'analyser le dataset métier sans écrire de code.

Dataset: 2 965 lignes, 26 colonnes (BigQuery ML datasets)

Les Données

Socio-démo

- sex
- education_level
- marital_status
- age

Financières

- limit_balance
- bill_amt_1 à bill_amt_6
- pay_amt_1 à pay_amt_6

Historique

- pay_0
- pay_2 à pay_6

Variable Cible : `default_payment_next_month (0/1)`

ATTENTION : La colonne `predicted_default_payment_next_month` ne doit **PAS** être utilisée comme feature. C'est une fuite de données (leakage) issue d'un modèle existant !

Étape 1 : Repo Git + EDA

01

Initialiser le dépôt Git (min. 3 commits propres, structure standard).

02

Créer un dictionnaire de données (définition, type, exemples pertinents).

03

Contrôle qualité strict : manquants, doublons, aberrants, cohérence logique.

04

Explorer la cible : calculer le taux de défaut global et par segments clés.

05

Définir le protocole d'évaluation : split, choix des métriques, et stratégie face au déséquilibre.

Étape 2 : Un modèle Production-Ready

Pipeline scikit-learn

Split strict, imputation, scaling/encodage intégrés au pipeline pour éviter toute fuite.

Traitement du déséquilibre

Utilisation et justification de class_weight, ou de techniques de sous/sur-échantillonnage.

Métriques adaptées

PR-AUC (prioritaire sur ROC-AUC), Recall, Precision, analyse de la matrice de confusion.

Modélisation itérative

Baseline simple (LogisticRegression) puis modèle amélioré (ex: XGBoost/LightGBM).

Choix du seuil

Définir un top K% à relancer ou un seuil de probabilité aligné sur une logique de coût métier.

Livrables techniques

Sauvegarde du modèle et génération du fichier de scoring (id, proba_default, label_pred).

Étape 3 : POC GenAI "Talk to my Data"

Un assistant d'analyse en langage naturel.

L'utilisateur pose une question en français l'agent génère du code Python (pandas) l'exécute sur le DataFrame en mémoire affiche la réponse formatée.

Stack Technique

LangChain v1
Streamlit

Contraintes Strictes

Pas de SQL
Pas d'accès réseau
Pas d'écriture disque

Architecture des Agents

ProfilingAgent

"Quel est le taux de défaut global ?"

Action attendue : describe(), value_counts(), graphiques de base (hist).

SegmentationAgent

"Taux de défaut par tranche d'âge ?"

Action attendue : groupby(), tableau trié + génération de graphique.

ComparisonAgent

"Comparer défaut vs non-défaut sur limit_balance"

Action attendue : calculs de moyennes segmentées, boxplots.

ModelInsightAgent

"Quelles features sont les plus importantes ?"

Action attendue : lecture des feature importances, consultation des perfs de test.

Travail attendu sur le POC

- **Agent LangChain v1**

  Développement de l'agent avec un outil d'exécution Python contrôlée (agissant sur le DataFrame chargé en mémoire).
- **Format de sortie imposé**

  L'interface doit afficher : La réponse en FR + Le code Python exécuté + Le résultat (table/graphique).
- **Golden Set de Validation**

  Constitution d'un set de 10+ questions pour valider l'agent (vérification manuelle vs calculs hors agent).
- **Application Streamlit**

  Champ de question, affichage structuré, et maintien d'un historique de conversation basique.
- **Gestion des refus**

  L'agent doit répondre "impossible avec les données disponibles" si la question sort du périmètre du dataset.

Livrables & GitHub

```
projet-ia/
├── app/
│   ├── agents/
│   │   ├── prompt.py
│   │   ├── tools.py
│   │   └── agent.py
│   └── streamlit_app.py
├── data/raw/
├── models/
│   └── model.joblib
├── notebooks/
│   ├── 01_EDA.ipynb
│   ├── 02_baseline.ipynb
│   └── 03_advanced.ipynb
├── src/
│   ├── config.py
│   ├── data_prep.py
│   └── ...
├── reports/
├── requirements.txt
└── README.md
```

Dépôt Git complet
Notebooks EDA & ML
Modèle (joblib) + script inférence
POC GenAI (Streamlit + LangChain)
Rapport synthétique (métriques, seuil, limites)

Grille d'Évaluation

Cadrage & traçabilité

Repo propre, reproductible, commits lisibles.

Analyse (EDA)

Contrôles qualité, analyses pertinentes, compréhension de la cible.

Pipeline ML

Pas de leakage, validation rigoureuse, métriques liées au déséquilibre.

Modèle & Décision

Performance, calibration, choix de seuil/topK justifié métier.

POC GenAI

Implémentation LangChain, affichage code, robustesse, gestion des refus.

Qualité générale

Lisibilité, factorisation, commentaires, clarté du README.

De la modélisation ML à l'IA générative : un projet complet.

Questions ?